

Developer Integration Checklist — Shopify Handoff

Pre-Integration Setup

- [] Clone repository and run `npm install`
- [] Copy `.env.example` to `.env`
- [] Verify prototype runs: `npx remix vite:dev --port 3000`
- [] Read `docs/SHOPIFY_INTEGRATION_GUIDE.md`
- [] Read `ARCHITECTURE.md` and `ASSIGNMENT_LOGIC.md`

Phase 1: Shopify App Setup

- [] Create Shopify Partner account (if not existing)
- [] Create a new app in Shopify Partner Dashboard
- [] Configure OAuth redirect URLs
- [] Set `SHOPIFY_API_KEY` and `SHOPIFY_API_SECRET` in `.env`
- [] Install `@shopify/shopify-api` package
- [] Replace mock client in `shopify-client.server.js` with real client
- [] Test OAuth flow with development store
- [] Set `SHOPIFY_ACCESS_TOKEN` after successful installation

Phase 2: Database Migration

- [] Set up PostgreSQL database (local or cloud)
- [] Update `prisma/schema.prisma` datasource to PostgreSQL
- [] Update `DATABASE_URL` in `.env`
- [] Run `npx prisma migrate dev --name init`
- [] Verify schema matches all models (including Phase 2.5 additions)
- [] Update `prisma/seed.js` to work with real SKU data
- [] Run seed script: `npx prisma db seed`
- [] Test Prisma queries in `data-access.server.js`

Phase 3: Product Sync

- [] Implement `fetchProducts()` in `shopify-products.server.js`
- [] Handle pagination (Shopify uses cursor-based pagination)
- [] Map Shopify product fields to local Product model
- [] Store `shopifyProductId` and `shopifyVariantId`
- [] Implement `shopifyProductToLocal()` mapping function
- [] Implement `fetchInventoryLevels()` for stock tracking
- [] Implement `decrementInventory()` for post-shipment
- [] Run initial product sync from Shopify to database

- [] Verify product data matches SKU mapping CSV
- [] Test assignment engine with synced product data

Phase 4: Customer Integration

- [] Implement `fetchCustomer()` in `shopify-customers.server.js`
- [] Implement `fetchSubscribedCustomers()`
- [] Create metafield definitions in Shopify (run `createMetafieldDefinitions()`)
- [] Implement `fetchCustomerMetafields()` and `updateCustomerMetafield()`
- [] Implement `shopifyCustomerToLocal()` mapping
- [] Write migration script to populate customer metafields from existing data
- [] Test customer sync with development store

Phase 5: Subscription API

- [] Implement `fetchSubscriptionContract()` in `shopify-subscriptions.server.js`
- [] Implement `createSubscription()` with draft → commit workflow
- [] Implement `pauseSubscription()` and `resumeSubscription()`
- [] Implement `cancelSubscription()`
- [] Implement `updateSubscriptionLineItem()` for assignment engine
- [] Create selling plan groups for each collection type
- [] Test subscription creation flow end-to-end
- [] Test pause/resume with grandfathering calculation

Phase 6: Order Management

- [] Implement `createDraftOrder()` in `shopify-orders.server.js`
- [] Implement `completeDraftOrder()` for post-billing
- [] Implement `addFulfillment()` for tracking
- [] Implement `fetchCustomerOrders()` for history
- [] Test order creation → fulfillment → delivery flow
- [] Verify collection records update on delivery

Phase 7: Webhook Integration

- [] Implement HMAC validation in `validateWebhookHmac()`
- [] Register all webhooks via `registerWebhooks()`
- [] Implement each webhook handler:
 - [] `products/update` → sync product data
 - [] `products/delete` → archive product
 - [] `inventory_levels/update` → update stock, trigger OOS shift
 - [] `customers/create` → create local record
 - [] `customers/update` → sync data
 - [] `customers/delete` → archive and cancel
 - [] `orders/create` → link to shipment
 - [] `orders/paid` → confirm payment

- [] `orders/fulfilled` → update tracking, notify customer
- [] `subscription_contracts/create` → run initial assignment
- [] `subscription_contracts/update` → handle status changes
- [] `app/uninstalled` → cleanup
- [] Create Remix route for webhook endpoint (`/webhooks`)
- [] Test with Shopify CLI webhook forwarding
- [] Verify webhook event logging in `WebhookEventLog`

Phase 8: Route Updates

- [] Switch routes from `mock-db` to `data-access.server.js`:
- [] `app.cabinet._index.jsx` (customer dashboard)
- [] `app.cabinet.collection.jsx` (collection grid)
- [] `app.cabinet.missing.jsx` (missing elements)
- [] `app.cabinet.wishlist.jsx` (wishlist)
- [] `app.cabinet.subscription.jsx` (subscription management)
- [] `app.cabinet.preferences.jsx` (preferences)
- [] `app.cabinet.onboarding.jsx` (onboarding)
- [] `app.admin.operations.jsx` (admin operations)
- [] `app.admin.customers.jsx` (admin customers)
- [] `app.admin.customer.$id.jsx` (admin customer detail)
- [] `app.admin.analytics.jsx` (analytics)
- [] `app.admin.pricing.jsx` (pricing config)
- [] Remove `?customer=` query param hack (use real auth)
- [] Implement proper admin authentication

Phase 9: Notification System

- [] Choose email provider (SendGrid, Postmark, SES)
- [] Implement real email sending in `notifications.server.js`
- [] Create email templates for each notification type:
- [] Assignment preview
- [] Shipment confirmation
- [] Restock alert
- [] Discount alert
- [] Billing reminder
- [] Collection milestone
- [] Pause confirmation
- [] Welcome
- [] Test email delivery
- [] Set `FEATURE_EMAIL_NOTIFICATIONS=true`

Phase 10: Testing & QA

- [] All routes return 200 OK

- ☐ Assignment engine works with real products
- ☐ Subscriptions create/pause/resume/cancel correctly
- ☐ Webhook handlers process events correctly
- ☐ Customer collection updates after order fulfillment
- ☐ Grandfathering calculations are correct
- ☐ Discount alerts trigger at correct thresholds
- ☐ Export CSV/JSON works with real data
- ☐ Search/filter works on admin pages
- ☐ Periodic table displays correct owned/available status
- ☐ Mobile responsive layout works

Phase 11: Deployment

- ☐ Set `APP_MODE=production` in deployment environment
- ☐ Set `NODE_ENV=production`
- ☐ Configure all production env vars
- ☐ Run database migrations
- ☐ Install app on production Shopify store
- ☐ Register production webhooks
- ☐ Run initial data sync
- ☐ Verify all features work in production
- ☐ Monitor logs for errors
- ☐ Set up error alerting (Sentry or similar)

Phase 12: Documentation

- ☐ Update README.md with production setup instructions
- ☐ Document any deviations from this checklist
- ☐ Create runbook for common operations
- ☐ Document backup/recovery procedures